

Computational Linguistics 1 (2026)

Assignment 4: Dependency Parsing

X-bar Schema, Universal Dependency, and Paninian Dependency

Deadline

April 23, 2026 (11:59 PM)

Objective

This assignment introduces students to three foundational frameworks for syntactic analysis:

- **X-bar Schema** – A phrase-structure theory that posits hierarchical internal structure for every phrase, with specifier, head, and complement positions.
- **Universal Dependencies (UD)** – A cross-linguistically consistent dependency annotation scheme that relates each word to a head via a typed arc.
- **Paninian Dependency (PD)** – A grammar formalism inspired by ancient Sanskrit grammarian Paṇini, which annotates relations in terms of *kāraṅgas* (semantic participant roles).

Students will draw trees for ten English sentences under each formalism, and then write a Python program that converts a Paninian dependency annotation to a Universal Dependency annotation.

Important Notes

- You must annotate **all 10 sentences** under all three formalisms.
- Trees may be drawn using any tool (`forest`, `tikz-dependency`, Brat, UD Annotatrix, or hand-drawn and scanned), but must be legible.
- For Task 2, submit a `.py` file. The skeleton provided must be preserved; do *not* rename functions.
- Late submissions will not be accepted at all.
- Collaboration on the conceptual design of trees is not permitted. Code may use standard libraries only (`conllu`, `nlTK`, `stanza`).
- Clearly label every dependency arc and every node label.

Sentences

The following ten sentences are used across all tasks. Sentences S9 and S10 are designed to test specific constructions (copula, passive, causative, coordinating conjunction) that interact with X-bar, UD, and Paninian annotation in non-trivial ways.

ID	Sentence
S1	The red apples were washed thoroughly, but the green apples were washed badly.
S2	Priya told the police that she was the perpetrator.
S3	Ram's tables are wooden.
S4	I am tired because I did not sleep well yesterday.
S5	Lazy men write assignments lazily.
S6	Bananas and guavas will be cut by Rahul for the fruit salad.
S7	He thought he could escape the prison.
S8	Men are from Mars and women are from Venus.
S9	The teacher made the students read the chapter.
S10	The cake was baked by Meera and decorated by her sister.

Table 1: Sentences for annotation

Task 1: Make Dependency Trees (60 Marks)

Task 1a: X-bar Schema (20 Marks)

For each of the 10 sentences draw the full X-bar tree.

Task 1b: Universal Dependency Tree (20 Marks)

Link for the tagset - <https://universaldependencies.org/u/dep/>

For each sentence, produce a dependency tree diagram using universal dependency relations.

Task 1c: Paninian Dependency Tree (20 Marks)

Draw the Paninian dependency tree for each sentence, using the kāraka tags.

Task 2: Convert Paninian to Universal Dependency (40 Marks)

For this task, you must write a Python script that programmatically converts a parsed Paninian Dependency tree into a Universal Dependency (UD) tree.

Strict I/O Format Specification:

Your script **must** accept exactly two command-line arguments: an input JSON file and an output JSON file.

```
python converter.py input.json output.json
```

Input Format (input.json): A JSON array of objects representing nodes. Each object contains `id`, `word`, `head_id`, and `paninian_relation`.

Output Format (output.json): The exact same JSON array, but every object must now include a new key called `ud_relation` containing your mapped tag.

Code Skeleton:

```
1 import sys
2 import json
3
4 def map_relation(paninian_tag):
5     """
```

```
6 Maps a single Paninian dependency tag to a UD tag.
7 Returns 'dep' as a fallback for unknown/unmapped tags.
8 """
9 # TODO: Define your mapping logic here
10 mapping_dict = {
11     # 'k1': 'nsubj',
12 }
13 return mapping_dict.get(paninian_tag, 'dep')
14
15 def process_file(input_path, output_path):
16     """
17     Reads the input JSON, applies the mapping, and writes to output JSON.
18     """
19     with open(input_path, 'r', encoding='utf-8') as f:
20         tree_data = json.load(f)
21
22     for node in tree_data:
23         # Extract the Paninian tag and find the UD equivalent
24         p_tag = node.get('paninian_relation', '')
25         node['ud_relation'] = map_relation(p_tag)
26
27     with open(output_path, 'w', encoding='utf-8') as f:
28         json.dump(tree_data, f, indent=4)
29
30 if __name__ == "__main__":
31     # Do NOT modify this execution block
32     if len(sys.argv) != 3:
33         print("Usage: python converter.py <input.json> <output.json>")
34         sys.exit(1)
35
36     input_file = sys.argv[1]
37     output_file = sys.argv[2]
38     process_file(input_file, output_file)
```

Evaluation Scheme

- **Task 1 (60 Marks Total):**
 - 2 Marks per correctly constructed X-bar tree (20 Marks).
 - 2 Marks per correctly constructed UD tree (20 Marks).
 - 2 Marks per correctly constructed Paninian tree (20 Marks).
- **Task 2 (40 Marks Total):**
 - Mapping Dictionary Completeness (15 Marks): Correctly identifying and translating core arguments, adjuncts, and modifiers.
 - Code Logic and Execution (15 Marks): The script runs without syntax errors and correctly processes the input schema.
 - Edge Cases & Fallbacks (10 Marks): Handling of unmapped tags, root nodes, and complex relations.

Submission Instructions

Submit a ZIP file named:

`<rollnumber>_assignment4.zip`

The ZIP must contain the following structure exactly:

- Assignment4_Answers.pdf: A single PDF document containing your hand-drawn or digitally created syntactic trees for Task 1. Ensure the images are clear and legible.
- converter.py: Your Python script containing the implementation for Task 2.
- README.md (Optional): Any assumptions you made regarding tag mappings or dependencies.

Academic Integrity

All submitted work must be your own. **Collaboration on tree annotations or code is not permitted.** You may discuss high-level concepts with classmates, but sharing trees, CoNLL-U tables, or source code - directly or via public repositories - constitutes academic misconduct and will be penalised as per institute policy. Automated plagiarism detection will be applied to all Task 2 submissions.